

Store Rating Platform

Project structure, schema and flow documentation

Stack: Express.js, PostgreSQL, React (Vite), JWT authentication. One login for three roles: System Administrator, Normal User, Store Owner.

1. Project Structure

```
store-rating/
|-- README.md
|-- backend/
|   |-- package.json, .env.example
|   |-- db/
|       |-- schema.sql      (tables, constraints, indexes)
|       |-- seed.js         (default admin)
|   |-- src/
|       |-- server.js        (app bootstrap, CORS, error handler)
|       |-- config/db.js     (pg Pool)
|       |-- middleware/auth.js (authenticate, authorize(roles))
|       |-- utils/validators.js (express-validator rules)
|       |-- utils/query.js    (safe sort + filter builders)
|       |-- controllers/
|           |-- authController.js (signup, login, updatePassword)
|           |-- adminController.js (dashboard, users, stores)
|           |-- userController.js (store list, rate store)
|           |-- ownerController.js (store dashboard)
|       |-- routes/index.js
|-- frontend/
|   |-- package.json, vite.config.js, index.html
|   |-- src/
|       |-- main.jsx, App.jsx, styles.css, hooks.js, utils.js
|       |-- api/client.js      (axios + JWT interceptor)
|       |-- context/AuthContext.jsx
|       |-- components/ Navbar, ProtectedRoute, SortableTable, FormField
|       |-- pages/ Login, Signup, ChangePassword,
|           AdminDashboard, UserStores, OwnerDashboard
```

2. Database Schema

Table	Columns	Constraints / Indexes
users	id, name, email, password, address, role, created_at	email UNIQUE; name length >= 20; role enum ADMIN/USER/OWNER; index on role
stores	id, name, email, address, owner_id, created_at	email UNIQUE; owner_id FK to users (ON DELETE SET NULL); index on owner_id
ratings	id, user_id, store_id, rating, created_at, updated_at	rating CHECK 1-5; UNIQUE(user_id, store_id) so one rating per user per store; FKs ON DELETE CASCADE; index on store_id

Relationships: a Store Owner (user) owns zero or one store; a store has many ratings; a normal user has many ratings (one per store). Average store rating is computed with AVG(rating) at query time, so it is always current.

3. API Endpoints

Method	Path	Role	Description
POST	/api/auth/signup	Public	Register normal user
POST	/api/auth/login	Public	Login, returns JWT + user
PUT	/api/auth/password	Any	Change own password
GET	/api/admin/dashboard	Admin	Total users, stores, ratings
GET	/api/admin/users	Admin	List; filters name/email/address/role; sortBy, order
POST	/api/admin/users	Admin	Create user of any role
GET	/api/admin/users/:id	Admin	Details (owner shows rating)
GET	/api/admin/stores	Admin	List with rating; filters + sort
POST	/api/admin/stores	Admin	Create store (optional owner)
GET	/api/stores	User	Stores with overall + own rating; search name/address
PUT	/api/stores/:storeId/rating	User	Submit or modify rating (upsert)
GET	/api/owner/dashboard	Owner	Average rating + users who rated

4. Project Flow

Authentication flow

```
Browser          React          Express          PostgreSQL
| submit login form -> validate -> POST /auth/login -> validators -> SELECT user
|                                     bcrypt.compare
|                                     <- {token, user} <----- sign JWT (id, role, name)
| store token (localStorage), set AuthContext
| redirect by role: ADMIN -> /admin, USER -> /stores, OWNER -> /owner
| every request: axios adds Authorization: Bearer <token>
| server: authenticate (verify JWT) -> authorize(role) -> controller
| 401 response -> token cleared, redirect to /login
```

Role flows

Role	Flow
Normal User	Sign up (validated) -> auto login -> /stores -> search by name/address, sort -> pick 1-5 and Submit; later press Modify to change (upsert on UNIQUE(user_id, store_id)) -> overall rating refreshes.
Admin	Login -> /admin dashboard shows totals -> add user / add store forms -> Users and Stores tabs with column filters and sortable headers -> click a user row for details (owners show average rating).
Store Owner	Login -> /owner -> sees store average rating, total ratings and sortable table of users who rated -> can change password.

Rating submit sequence

```
UserStores page -> PUT /stores/:id/rating {rating}
-> authenticate -> authorize('USER') -> validate 1..5
-> check store exists
-> INSERT ... ON CONFLICT (user_id, store_id) DO UPDATE SET rating, updated_at
-> 200 -> frontend reloads list (new average + myRating)
```

5. Validation Rules

Field	Rule
Name	20 to 60 characters
Address	Max 400 characters
Password	8 to 16 characters, at least 1 uppercase and 1 special character
Email	Standard email format
Rating	Integer 1 to 5

Rules are enforced twice: in React forms for instant feedback, and in express-validator plus DB CHECK constraints for integrity.

6. Best Practices Applied

Layered backend (routes, controllers, middleware, utils); bcrypt password hashing; JWT with role-based authorization on server and client; parameterised queries and whitelisted sort columns; normalized schema with FKs, unique and check constraints, and indexes; server-side sorting and filtering; centralized error handling; environment-based configuration.

7. Run Instructions

```
createdb store_rating && psql -d store_rating -f backend/db/schema.sql
cd backend && cp .env.example .env && npm install && npm run seed && npm run dev
cd frontend && cp .env.example .env && npm install && npm run dev
Default admin: admin@example.com / Admin@123
```